ARQUITETURA DE SOFTWARE

TRABALHO EM GRUPO N1 - BOEING 737

Prof. Jefferson Salomão Rodrigues

Iago Cordeiro Felix

Gustavo Lucena de Aguiar

Hiago de Oliveira Gregorini

João Vitor Teixeira de Araújo

INTRODUÇÃO

A Arquitetura de Software tem um papel importante no desenvolvimento de sistemas críticos, principalmente na aviação, onde falhas podem ter consequências graves. Por isso, é fundamental que esses sistemas sejam projetados com foco em segurança, confiabilidade e prevenção de falhas.

Este trabalho analisa os problemas relacionados ao sistema MCAS da aeronave Boeing 737 MAX 8, que esteve envolvido em acidentes de grande repercussão. A partir disso, são discutidas as falhas no sistema e apresentada uma proposta de melhoria com base em boas práticas de desenvolvimento de software.

EXPLICAÇÃO DO PROBLEMA

Contexto histórico do desastre:

O avião Boeing 737 MAX 8 veio em resposta para competir com os novos lançamentos da Airbus, cortando custos de produção para lançar o mais rápido possível, a Boeing decidiu lançar seu novo modelo baseado em um já consolidado no mercado, podendo assim, investir menos em burocracia e em treinamento dos pilotos. Por conta da necessidade de reduzir custos e por motores, mesmo que mais eficientes, porém, maiores, foi necessário que posicionassem ele mais à frente das asas, o que causou uma bola de neve no projeto, e ocasionando um Stall no avião, que é a perda de sustentação da nave devido ao nariz do avião estar muito elevado, atingindo um ângulo de ataque crítico, o que pode levar à perda de controle da

aeronave. Para solucionar esse problema, a Boeing criou o MCAS (Sistema de Aumento de Características de Manobra).

Impactos financeiros, humanos, sociais e/ou ambientais:

Esse problema acarretou dois acidentes com esse modelo causando 346 mortes, um voo doméstico pela Indonésia Lion Air com 189 mortes 13 minutos após a decolagem, e outro pela Ethiopian Airlines causando 157 mortes e a queda após seis minutos da decolagem. O que levou ao aterramento dessa aeronave em 18 de março de 2019. Os acidentes e o aterramento custaram à Boeing cerca de US$ 20 bilhões em multas, indenizações e honorários legais, com perdas indiretas de mais de US$ 60 bilhões em 1200 pedidos cancelados.

Papel do sistema de software ou sistema sociotécnico envolvido:

Do ponto de vista da Arquitetura de Software, o problema principal está na presença de um Single Point of Failure, já que o sistema dependia apenas de um sensor, faltando uma validação cruzada entre os dois. Além da ausência de mecanismos fail-safe, permitindo que o sistema continuasse operando mesmo diante de dados inconsistentes.

ANÁLISE DO PROBLEMA SOB A PERSPECTIVA DA ARQUITETURA DE SOFTWARE

O sistema funcionava com uma arquitetura muito simplificada e com limitações em questões de segurança, embora o avião tenha 2 sensores de angulo de ataque, o sistema MCAS verificava apenas um deles por vez. O problema está na verificação de apenas um sensor, e na falta de transparência e treinamento dos pilotos. Isso causa:

Single point of failure: Um único ponto no sistema que causa a falha completa no sistema

Falta de redundância: O sistema não fazer verificação nos cruzada entre os dois sensores, dando falsos positivos.

Falta de transparência: Os pilotos não foram notificados nem treinados sobre o MCAS, causando confusão e falta de preparo para situações quando acontecia um stall e o sistema agia de forma errada.

Alto acoplamento: A dependência forte apenas nos sensores, a falta de um passo extra de verificação dos dados coletados por eles.

Todos esses problemas em conjunto levaram a falha que foi o MCAS e seus acidentes causando centenas de mortes e bilhões em prejuízos financeiros

MODELAGEM UML

DER:

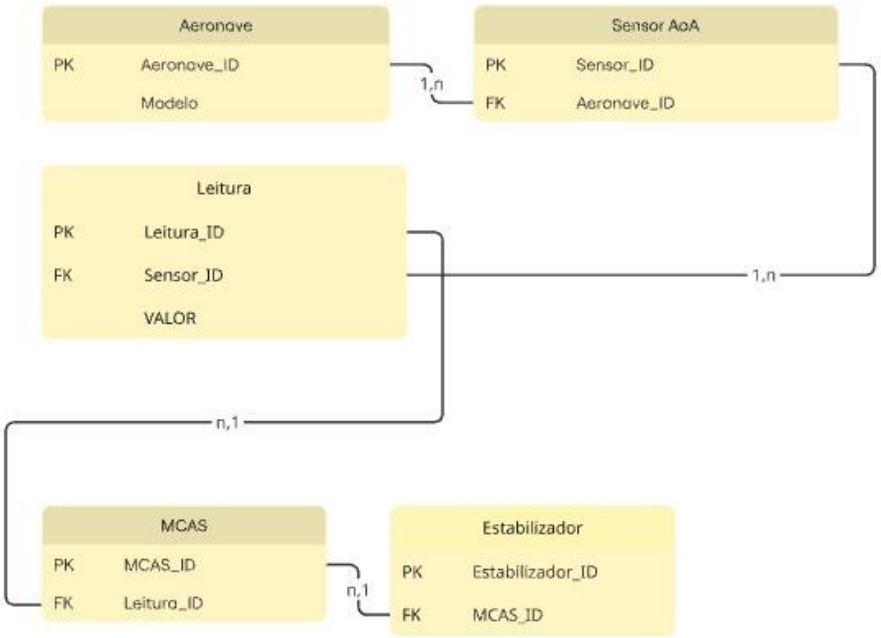

Figura 1 – Diagrama Entidade-Relacionamento

O diagrama mostra uma cadeia de dependência:

- A Aeronave possui um Sensor.
- Sensor gera uma Leitura.
- O MCAS analisa essa Leitura.
- O MCAS envia um comando para o Estabilizador (movendo o nariz do avião para baixo, se necessário)

Diagrama de Caso de Uso:

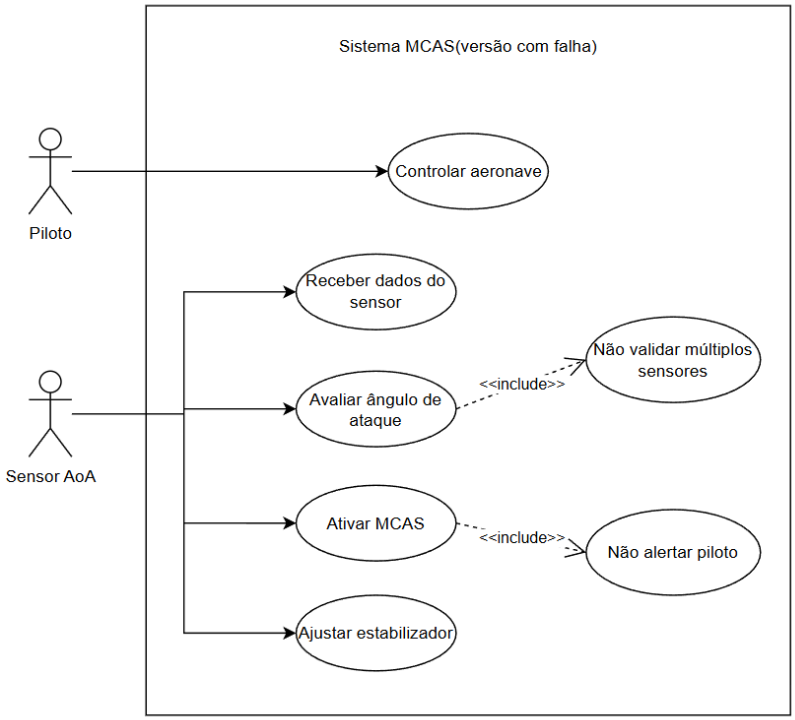

Figura 2 – Diagrama de Caso de Uso

- Atores: O Piloto e o Sensor AoA.
- A Falha Crítica (<<include>>): O diagrama destaca onde o software errava por design:
- Ao Avaliar ângulo de ataque, o sistema obrigatoriamente (include) passava por "Não validar múltiplos sensores". Ou seja, ele ignorava o segundo sensor.
- Ao Ativar MCAS, ele obrigatoriamente passava por "Não alertar piloto", deixando a tripulação no escuro sobre o que estava acontecendo.
- O Conflito: O Piloto tenta "Controlar a aeronave" enquanto o sistema, simultaneamente, tenta "Ajustar estabilizador" com base em dados não validados.

Diagrama de Classes:

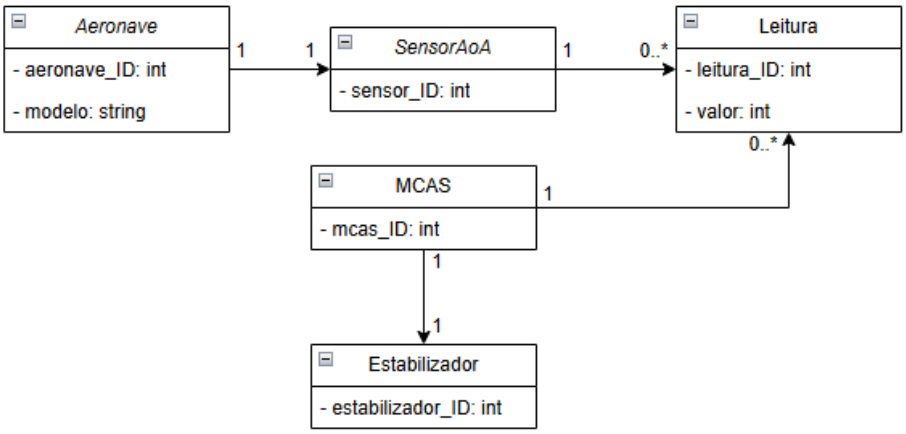

Figura 3 – Diagrama de Classes

- Atributos: Cada classe mostra o tipo de dado (ex: aeronave_ID: int, modelo: string).
- Relacionamentos (Associações):
- Aeronave (1) — (1) SensorAoA: Indica que o sistema foi projetado para que cada aeronave fosse lida por apenas um sensor específico naquele contexto de software.
- SensorAoA (1) — (0..*) Leitura: Um sensor gera várias leituras ao longo do tempo.
- MCAS: É a classe central que conecta a Leitura ao Estabilizador. Ele "consome" a leitura e "comanda" o estabilizador.

Diagrama de Atividades:

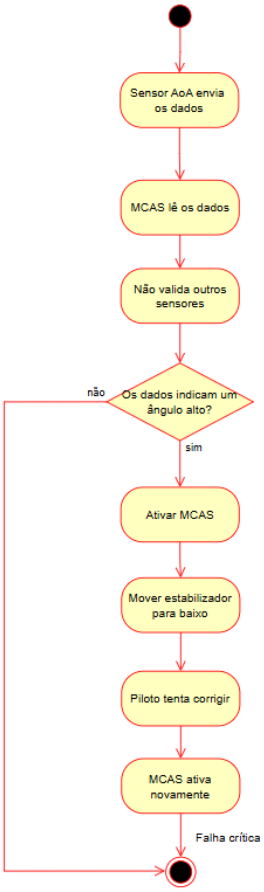

Figura 4 – Diagrama de Atividades

- Entrada de Dados e Falha de Validação: O ciclo começa com o Sensor AoA enviando dados para o software. Imediatamente, ocorre o erro de design mais grave: o sistema não valida outros sensores, aceitando a informação de uma única fonte como verdade absoluta.
- Tomada de Decisão: O sistema avalia se o ângulo de ataque é considerado "alto" (risco de estol).
- Se não, o fluxo se encerra normalmente.
- Se sim, o MCAS é ativado automaticamente para "salvar" o avião.
- Ação Mecânica e Conflito: Uma vez ativo, o sistema move o estabilizador para baixo de forma forçada. O diagrama mostra que, quando o piloto tenta corrigir a trajetória puxando o nariz para cima, o sistema entra em um loop, ignorando o comando manual e ativando o MCAS novamente.
- Desfecho: O fluxo termina em uma Falha Crítica, demonstrando que a repetição sucessiva de correções automáticas erradas, sem validação de dados ou intervenção de segurança, tornava a queda da aeronave inevitável.

ANÁLISE DE REQUISITOS

Requisitos Faltantes

FUNCIONAIS:

RF01 - Validação de Sensores: O software deve ser capaz de comparar e validar os dois sensores de ângulo de ataque disponíveis na aeronave para poder fazer uma comparação cruzada para tomar a decisão correta.

NÃO FUNCIONAIS:

RNF01 - Tolerância a Falhas (Redundância): O sistema não deve possuir um Single Point of Failure (Ponto Único de Falha). Decisões críticas de voo devem ser baseadas em validação cruzada de múltiplos sensores.

RNF02 - Disponibilidade: Os dados críticos de voo devem estar disponíveis em tempo real para o processamento do MCAS durante todas as fases da decolagem e cruzeiro.

RNF03 - Um fail-safe, no caso de inconsistência de dados, o sistema deve desativar e alertar os pilotos.

A falta desses requisitos fez com que o sistema dependesse de apenas um sensor de ângulo de ataque, caracterizando o Single Point of Failure, além de não ter a disponibilidade dos dados para os pilotos, o que contribuiu diretamente para o acidente.

Requisitos Mal Priorizados

A priorização de requisitos relacionados à redução de custos e tempo de lançamento em detrimento de requisitos de segurança e confiabilidade comprometeu a arquitetura do sistema.

A decisão de minimizar a necessidade de treinamento dos pilotos e acelerar a certificação levou à adoção de soluções arquiteturais simplificadas, como a ausência de validação cruzada de sensores, aumentando significativamente o risco operacional

PROPOSTA DE SOLUÇÃO ARQUITETURAL (VISÃO 2026)

Estilo Arquitetural

Foi adotada a arquitetura **N-Tier**, que organiza o sistema em camadas bem definidas: sensores, processamento e apresentação. Essa separação permite reduzir o acoplamento entre os componentes e melhorar a manutenção do sistema. A camada de processamento é responsável pela validação dos dados, controle do MCAS e aplicação de mecanismos de segurança, enquanto a camada de apresentação garante a comunicação com o piloto por meio de alertas. Essa abordagem aumenta a confiabilidade e a tolerância a falhas, resolvendo problemas como o ponto único de falha presentes no sistema do Boeing 737 MAX 8.

Modelagem Da Solução

Diagrama N-Tier

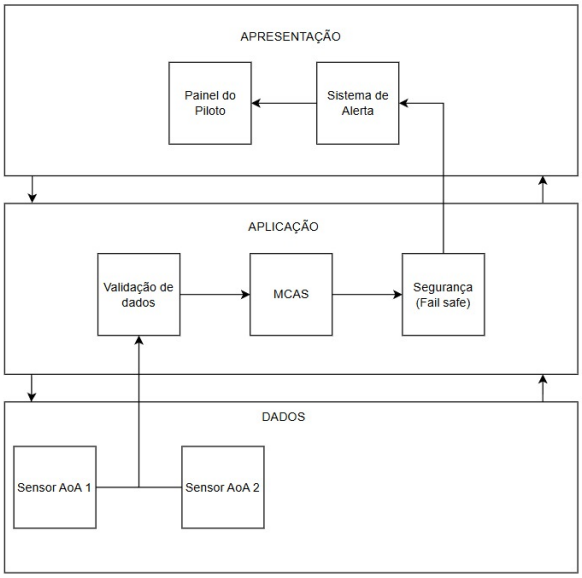

Figura 5 – Diagrama N-Tier

- Camada de Dados (Base): É onde tudo começa. Diferente do sistema original, aqui temos o Sensor AoA 1 e o Sensor AoA 2. Esta camada é responsável por captar os sinais físicos do ambiente e enviá-los para processamento.
- Camada de Aplicação (Cérebro): É onde a lógica de segurança acontece.
- Os dados dos dois sensores passam primeiro pela Validação de dados.
- O MCAS só processa a informação se ela for validada.

- Existe agora um módulo de Segurança (Fail safe): se algo estiver errado, ele impede ações catastróficas e envia um comando para a camada superior.
- Camada de Apresentação (Interface): É o nível de interação com o humano.
- O Sistema de Alerta recebe notificações da camada de segurança.
- O Painel do Piloto exibe as informações finais, garantindo que o piloto saiba exatamente o que o sistema está fazendo ou se há uma divergência nos sensores.

Diagrama De Classes:

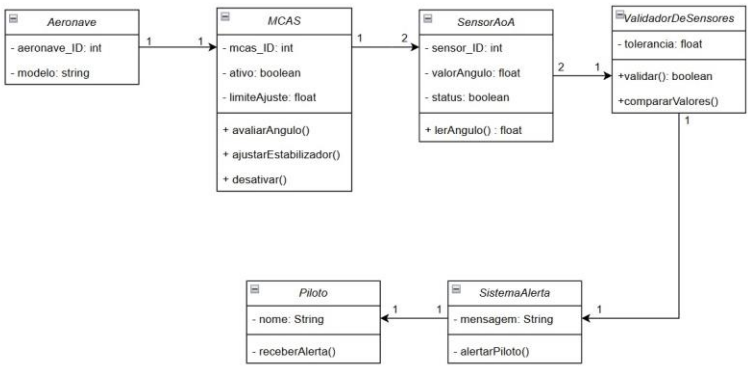

Figura 6- Diagrama de Classes

- Redundância de Entrada: O sistema agora exige a presença de dois sensores AoA (multiplicidade 2), eliminando o risco de um único sensor defeituoso enganar o software.
- Camada de Verificação (Validador): A introdução da classe ValidadorDeSensores cria um "filtro". Ela compara os valores de ambos os sensores e só autoriza a operação se a diferença entre eles estiver dentro de uma tolerancia aceitável.
- Controle e Limitação do MCAS: A classe MCAS ganha novos atributos de controle, como ativo (permitindo o desligamento do sistema) e limiteAjuste, que impede que o software aplique comandos de profundidade excessivos.
- Transparência com o Operador: Diferente da versão anterior, este modelo conecta o sistema a um SistemaAlerta que comunica falhas ou divergências diretamente ao Piloto, garantindo que a tripulação mantenha a consciência situacional.

Design Patterns

Para melhorar a estrutura e a segurança do sistema, foram utilizados alguns padrões de projeto:

SINGLETON

O padrão Singleton garante que exista apenas uma instância do MCAS no sistema.Isso evita conflitos e garante que o controle da aeronave seja centralizado.

OBSERVER

O Observer é usado no sistema de alerta. Quando há alguma mudança relevante (como divergência entre sensores), o sistema notifica automaticamente o piloto.

Isso garante comunicação em tempo real e evita que o piloto fique sem informação, como aconteceu no sistema original.

9. IMPLEMENTAÇÃO DA SOLUÇÃO (SIMULAÇÃO)

A implementação parcial da solução arquitetural proposta foi desenvolvida em um formato de simulação de software. Para validar a resiliência das melhorias propostas na arquitetura do sistema, o projeto foi construído materializando a abordagem em camadas (N-Tier).

A estrutura do sistema opera em tempo real e consolida a divisão de responsabilidades da seguinte forma:

- **Camada Física (Sensores):** A classe `SensorAOA` representa os sensores físicos de Ângulo de Ataque. O sistema instancia dois sensores independentes para fornecer redundância de entrada e eliminar o Ponto Único de Falha (*Single Point of Failure*). As leituras recebem uma injeção de leve ruído algorítmico para simular as variações do mundo real.
- **Camada de Validação (Fail-Safe):** A classe `ValidadorDeSensores` atua como um filtro de verificação, garantindo a validação cruzada antes de autorizar processamentos críticos. Ela avalia a diferença absoluta entre as leituras dos dois sensores. Se a divergência ultrapassar a tolerância máxima aceitável (estipulada em 5 graus), o sistema aciona o modo *Fail-Safe*, desativando o software.
- **Camada de Aplicação (Processamento):** A classe `MCAS` possui o controle de ativação e atua sobre o estabilizador da aeronave. Para evitar conflitos de controle no sistema, ela foi desenvolvida utilizando o padrão de projeto *Singleton*, o que garante a existência de apenas uma instância controlando o

nariz do avião. Além disso, o software agora impõe um limite estrito de atuação (ajuste máximo de 2.5 graus) para impedir comandos de profundidade excessivos.

- **Camada de Apresentação (Interface):** A classe `SistemaAlerta` gerencia a comunicação visual e situacional com a tripulação. Utilizando o padrão de projeto *Observer*, o sistema ouve e exibe automaticamente os alertas gerados (em destaque visual vermelho) caso haja qualquer anomalia ou desativação.

9.1 Cenários de Teste Simulados O script principal da simulação (`main.py`) valida a eficácia da arquitetura através de três cenários dinâmicos de voo:

1. **Voo Normal e Equilibrado:** Ambos os sensores captam dados normais e similares. A validação cruzada aprova as leituras e o MCAS permanece inativo.
2. **Ângulo Crítico (Stall Real Detectado):** A aeronave entra em um ângulo de ataque elevado. Ambos os sensores detectam a subida perigosa, o validador atesta a veracidade da leitura, e o MCAS atua de forma controlada para empurrar o estabilizador para baixo, salvando o avião.
3. **Falha de Sensor (Prevenção de Acidente):** Um dos sensores apresenta defeito e começa a enviar dados absurdos de um *Stall* inexistente, enquanto o segundo sensor mostra um voo normal. A camada de validação detecta imediatamente a discrepância, bloqueia a atuação do MCAS e alerta o piloto, prevenindo uma queda injustificada (o exato problema que causou as quedas originais).

9.2 Repositório e Disponibilização do Código Garantindo a rastreabilidade e a transparência do versionamento, todo o código-fonte pertinente a esta simulação foi publicado de maneira integral em um repositório público.

Link de Acesso ao Repositório da Simulação: https://github.com/JoaoVitor0106/Boieng737

DISCUSSÃO E CONCLUSÃO

A análise do sistema evidenciou que decisões arquiteturais inadequadas podem gerar consequências extremamente graves, especialmente em sistemas críticos como os da aviação. O caso do Boeing 737 MAX 8 demonstrou como a ausência de redundância, validação de dados e transparência pode levar a falhas catastróficas. A proposta apresentada corrige esses problemas ao introduzir validação cruzada de sensores, mecanismos de fail-safe e comunicação clara com o piloto.

Além disso, o uso de boas práticas de arquitetura e padrões de projeto contribui para um sistema mais seguro, confiável e preparado para lidar com situações inesperadas. Dessa forma, fica evidente a importância de priorizar requisitos de segurança e qualidade no desenvolvimento de software, principalmente em sistemas onde vidas humanas estão envolvidas.

REFERÊNCIAS

Aterramento do Boeing 737 MAX. **Wikipedia**. Disponível em: https://pt.wikipedia.org/wiki/Aterramento_do_Boeing_737_MAX.

TRAVIS, Gregory. How the Boeing 737 Max Disaster Looks to a Software Developer. **IEEE Spectrum**, 2019. Disponível em: https://spectrum.ieee.org/how-the-boeing-737-max-disaster-looks-to-a-software-developer. Acesso em: [Data de hoje].

HERNDON, J. M., et al. The Boeing 737 MAX: Lessons for Engineering Ethics. **Science and Engineering Ethics**, v. 26, n. 6, p. 2939-2954, 2020. Disponível em: https://www.ncbi.nlm.nih.gov/pmc/articles/PMC7351545/.

IEEE COMPUTER SOCIETY. "Lessons Must Be Learned - But Are They?". **IEEE Software**, 2019. Disponível em: https://www.computer.org/csdl/magazine/so/2019/04/08738154/1aW6lxQgdVu.